Software Requirements Document

Authors: TJ Drape, Afonso Azevedo, Roman Zalewski, John Markham

1. Introduction and Purpose

We are building a secure, local-hosted, CLI notes app powered by AI. The user will have access to reading past notes, writing new notes, and summarizing an old note. The intended purpose of the program is to give users a minimalistic, encryption-secured notes app to track and write anything, without fear of their data falling into the wrong hands.

1.1 Acronyms, Terms and References

Acronym	Meaning
SRD	Software Requirements Document
SUD	System Under Development
HC	Host Computer
TSC	Terminal-Start Command
CLI	Command-Line Interface
AI	Artificial Intelligence
TT	Trace Tag

Reference	Source
Google AI Overview of Non-functional Requirements	Google, 2025

2. Business Objectives

We are creating the SUD to apply the software methods we have learned from SE300 to a project. We intend to perform the common software lifecycle methods through this project. Additionally, we wish to create a resume-worthy application as we compete for summer internships, showcasing our software engineering expertise.

3. User Personas or Roles

Privacy-conscious consumer: Our main user category, someone who distrusts the current note-taking solutions, prefers self-hosted applications due to privacy concerns with large tech companies or cloud-based software.

Application enthusiast: A secondary consumer category, someone who enjoys testing and using new technologies, and does not stick with a single product unless it truly solves a problem in their life.

4. Feature List

The SUD is a local, command-line-based secure notes application that allows users to create, view, append, delete, and summarize personal notes. All notes are encrypted before being stored on the host computer and are only decrypted in memory when displayed to the user. The application provides a menu-driven interface that guides the user through all available actions and validates all user input. The SUD also provides basic AI functionality, including note summarization and keyword analysis, while ensuring that plaintext note contents are never exposed outside of the application.

5. Use Cases

- *The SUD shall allow a user to run the SUD using a TSC (TT: T_01)*

- *The SUD shall allow a user to choose an action from the given action menu (TT: T_02)*
- *The SUD shall disallow invalid menu choices (TT: T_03)*
- *The SUD shall perform the desired action from the given action menu that the user chose (TT: T_04)*
- *The SUD shall never expose the plain-text contents of a file outside of the SUD (TT: T_05)*
- *The SUD shall disallow the user from saving a blank note (TT: T_06)*

6. User Requirements

- *The SUD shall be available online as a public GitHub repository (TT: T_07)*
- *The SUD shall be cloned into a directory on the HC (TT: T_08)*
- *The user shall be able to access the terminal on the HC (TT: T_09)*
- *The user shall be able to copy/paste (TT: T_10)*

7. Functional Requirements

- *The SUD shall allow the user to create a new note (TT: T_11)*
- *The SUD shall allow the user to summarize a written note (TT: T_12)*
- *The SUD shall allow the user to append to a written note (TT: T_13)*
- *The SUD shall allow the user to display a written note (TT: T_14)*
- *The SUD shall encrypt a user's written note prior to saving (TT: T_15)*
- *The SUD shall decrypt a user's saved note prior to displaying (TT: T_16)*
- *The SUD shall allow a user to summarize a chosen note (TT: T_17)*
- *The software shall perform keyword analysis on the note (TT: T_18)*
- *The software shall save the user's notes on the HC (TT: T_19)*
- *The software shall perform user confirmation if a user chooses to delete a note (TT: T_20)*
- *The software shall perform input validation for all user interactions (TT: T_21)*
- *The software shall display the user's chosen action (TT: T_22)*

- *The SUD shall allow the user to delete a saved note (TT: T_23)*
- *The SUD shall notify the user if the note exceeds the supported size for summarization (TT: T_24)*
- *The SUD shall notify the user if the note exceeds a supported file size (TT: T_25)*

8. Nonfunctional Requirements

- *The SUD shall require no more than 3 minutes to set-up (TT: T_26)*
- *The SUD shall require only HC storage (TT: T_27)*
- *The SUD shall require a password verification to access (TT: T_28)*
- *The SUD shall have a menu where user's type in their desired action (TT: T_29)*
- *The SUD shall use a key map of numbers to actions (like, press 1 for new note, press 2 for summary, etc.) (TT: T_30)*
- *The SUD shall require a user generated password of at least 8 characters for encrypting/decrypting notes (TT: T_31)*
- *The SUD shall operate fully offline without requiring an internet connection (TT: T_32)*

9. Interface Requirements

- *The HC shall have a terminal from which the software will run (upon TSC) (TT: T_33)*
- *The HC shall have a typing interface (TT: T_34)*

10. Performance Requirements

- *The SUD shall encrypt a file of size 1MB within 10 seconds (TT: T_35)*
- *The SUD shall decrypt a file of size 1MB within 10 seconds (TT: T_36)*

- *The SUD shall summarize a file of size 1MB within 15 seconds (TT: T_37)*
- *The SUD shall perform keyword analysis on a 10MB file in under 10 seconds (TT: T_38)*
- *The SUD shall begin running within 1 second after TSC (TT: T_39)*
- *The software shall handle notes up to 15MB for summarization and keyword analysis without crashing (TT: T_40)*
- *The SUD shall handle all database interactions in under 5 seconds (TT: T_41)*

11. Availability Requirements

- *The SUD shall be available regardless of the internet connection of the HC (TT: T_42)*

12. Safety Requirements

- *The SUD shall provide a safe cancel option at every destructive or irreversible step (e.g., confirmation of a delete note action or overwrite note action). (TT: T_43)*
- *The SUD shall not terminate unexpectedly on invalid input (TT: T_44)*
- *The SUD shall display an error message when given invalid input*
- *(TT: T_45)*

13. Security Requirements

- *The SUD shall derive encryption keys from a user provided password using PBKDF2 (Password-Based Key Derivation Function 2) (TT: T_46)*
- *The SUD shall only store the encryption password or key in an encrypted format anywhere on the HC (TT: T_47)*
- *The SUD shall not provide password recovery mechanisms (design choice for maximum security) (TT: T_48)*

- *The SUD shall encrypt all notes using AES-256 encryption before saving to the HC (TT: T_49)*
- *The SUD shall not expose file contents outside of the program (TT: T_50)*
- *The SUD shall display an error message if encryption fails (TT: T_51)*
- *The SUD shall display an error message if decryption fails (TT: T_52)*
- *The SUD shall have write permissions on the HC (TT: T_53)*
- *The SUD shall have read permissions on the HC (TT: T_54)*

14. Design and Implementation Constraints

N/A

15. External System Requirements

- *The HC shall have Python 3.10 or later installed (TT: T_55)*

16. Quality Assurance Requirements

- *We will review these requirements as we learn more about verification and validation, later in the semester with SE300*

17. Documentation Requirements

- *The SUD shall be delivered with a set-up manual called “instructions.txt”. (TT: T_56)*
- *The SUD shall have a well-documented README file detailing our development process (TT: T_57)*

18. Trace Tags

- *Each requirement will have a trace tag, numbered 1 to N, and will be appended to the end of functions that complete the requirement*
- *Each team member has a unique trace tag identifier (Roman = 1, Afonso = 2, TJ = 3, John = 4)*
- *The identifiers will be appended to the function names with first number denoting author, then T with an underscore and the following numbers denoting requirement trace tag*
- *Example: my_function_3_T_01(), written by TJ, satisfies requirement 01*

SE 300

Appendix

Requirements Analysis & Review

Notes from team review:

- Split non-atomic requirements into several atomic requirements
- Add feature-specific requirements for encryption/decryption
- Add user password requirements for encryption/decryption
- Add runtime requirements for database functionality
- Add non-functional requirements
- Rearrange requirements for better document flow
- Remove repeated requirements in feature list
- Describe product usage in feature list instead of creating formal requirements
- Move requirements from wrong domain sections

Review Driven Modifications:

- Expand feature list for secure notes application.
- Turn the feature list into a descriptive paragraph.
- Elaborate on encryption methods to be implemented in the SUD.
- Expand Non-functional requirement section
- Move several non-functional requirements into more suitable areas, like performance requirements.
- Move mislabeled requirements into appropriate sections, like from availability to security requirements.